



北京師範大學
BEIJING NORMAL UNIVERSITY

计算机图形学 实验报告

姓名 杨博文 学号 202311081015
院系 人工智能学院 专业 计算机科学与技术

2025 年 3 月 15 日

摘 要

计算机图形学的第二次实验，主要内容包括为立方体加入旋转、透视投影效果、深度信息，使其更具立体感。

目录

1 运行环境及运行方式	3
2 任务 1: 通过旋转展示三视图	3
2.1 任务要求	3
2.2 关键代码	3
2.3 结果图片	4
3 任务 2: 透视投影效果	6
3.1 任务要求	6
3.2 关键代码	6
3.3 结果图片	6
4 任务 3: 加入深度测试	8
4.1 任务要求	8
4.2 关键代码	8
4.3 结果图片	9

1 运行环境及运行方式

硬件为 Macbook Air, M2 芯片, 操作系统为 MacOS Sequoia 15.0。

IDE 为 Xcode Version16.0。

使用 GLFW+GLAD, 程序在 Xcode 内编译运行。

2 任务 1: 通过旋转展示三视图

2.1 任务要求

在实验一中任务 2/3 的基础上, 对于带颜色或纹理的自定义立方体, 通过旋转操作展示其三视图 (主视图、俯视图、左视图)

2.2 关键代码

```
const char *vertexShaderSource = "#version 330 core\n"
...
"uniform mat4 transform;\n"
...
"    gl_Position = transform * vec4(aPos, 1.0);\n"
...
float time = glfwGetTime();
glm::mat4 transform = glm::mat4(1.0f);
if (time < 3.0f)
    transform = glm::rotate(transform,
        glm::radians(90.0f * time / 3.0f), glm::vec3(0.0f, 1.0f, 0.0f));
else if(time < 6.0f)
    transform = glm::rotate(transform, glm::radians(90.0f),
        glm::vec3(0.0f, 1.0f, 0.0f));
else if(time < 9.0f)
    transform = glm::rotate(transform,
        glm::radians(90.0f * (9.0f - time) / 3.0f),
        glm::vec3(0.0f, 1.0f, 0.0f));
else if (time < 12.0f)
    transform = glm::rotate(transform,
        glm::radians(-90.0f * (time - 9.0f) / 3.0f),
        glm::vec3(1.0f, 0.0f, 0.0f));
else if (time < 15.0f)
```

```
transform = glm::rotate(transform,
    glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
else if (time < 18.0f)
    transform = glm::rotate(transform,
    glm::radians(90.0f * (time - 18.0f) / 3.0f),
    glm::vec3(1.0f, 0.0f, 0.0f));
else if (time < 30.0f)
    transform = glm::rotate(transform,
    glm::radians(90.0f * (time - 18.0f) / 3.0f),
    glm::vec3(0.0f, 1.0f, 0.0f));

unsigned int transformLoc =
    glGetUniformLocation(shaderProgram, "transform");
glUniformMatrix4fv(transformLoc, 1, GL_FALSE, glm::value_ptr(transform));
...
```

这里我们基于上次的代码继续添加部分。我们将四面体先向左旋转展示右侧面，然后回到正面，然后向下旋转展示俯视图，然后回到正面，再向左连转 360° 。

2.3 结果图片

以下是我们的结果三视图，很显然的虽然感觉有点奇怪，但可以看出是一个完整的 3D 图形了。（这里由于我们在上周的任务中已经开启了深度，所以图层上没有遮挡问题）。

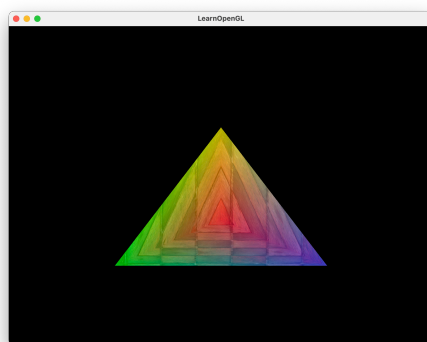


图 1: 实验 1 正视图

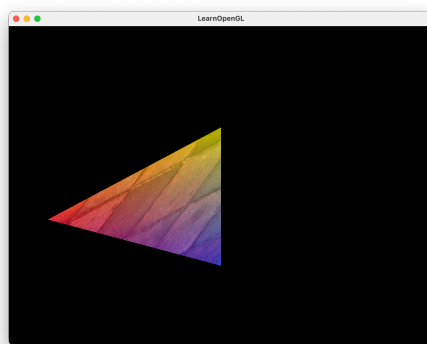


图 2: 实验 1 右视图

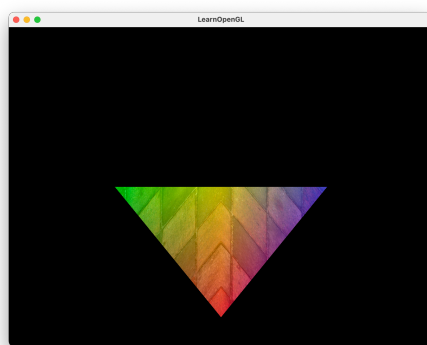


图 3: 实验 1 俯视图

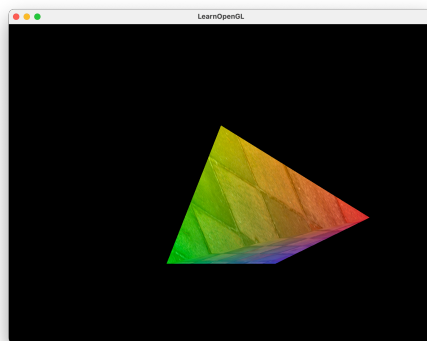


图 4: 实验 1 旋转过程中截图

3 任务 2: 透视投影效果

3.1 任务要求

在任务 1 的基础上，展示有透视投影效果的立方体三视图。

3.2 关键代码

```
const char *vertexShaderSource = "#version 330 core\n"
...
"uniform mat4 projection;\n"
"uniform mat4 view;\n"
...
"    gl_Position = projection * view * transform * vec4(aPos, 1.0);\n"
...
glm::mat4 projection = glm::perspective(glm::radians(45.0f),
-(float)SCR_WIDTH/SCR_HEIGHT, 0.1f, 100.0f);
unsigned int projectionLoc = glGetUniformLocation
(shaderProgram, "projection");
glUseProgram(shaderProgram);
glUniformMatrix4fv(projectionLoc, 1, GL_FALSE, glm::value_ptr(projection));

glm::mat4 view = glm::lookAt(
    glm::vec3(0.0f, 0.0f, -3.0f),
    glm::vec3(0.0f, 0.0f, 0.0f),
    glm::vec3(0.0f, 1.0f, 0.0f)
);
unsigned int viewLoc = glGetUniformLocation(shaderProgram, "view");
glUseProgram(shaderProgram);
glUniformMatrix4fv(viewLoc, 1, GL_FALSE, glm::value_ptr(view));
...
```

这里我们不仅使用了投影矩阵，还额外使用了观察矩阵来移动我们的摄像机（如果不动的话会发现我们看到一个巨大的立方体，就在我们眼前）。

3.3 结果图片

由于我们在上一次实验中就已经进行了深度处理，所以这部分代码使用的是将深度处理注释掉的情况，与任务 3 只有这么一点区别。故三视图的图片在任务三中：[图 9](#)、

图 10、图 11。这里我们放一张从背面看的图图 5，两张产生深度问题的图图 6、图 7，和一张正面正确的图图 8。

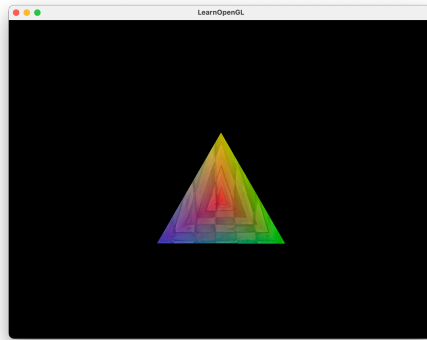


图 5: 实验 2 背面图

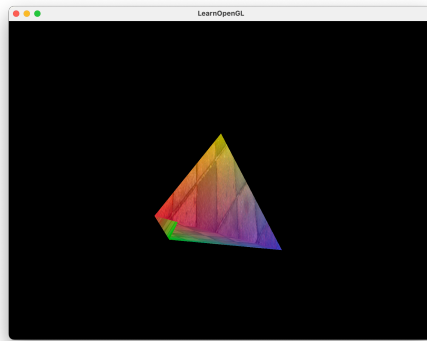


图 6: 实验 2 深度问题图 1

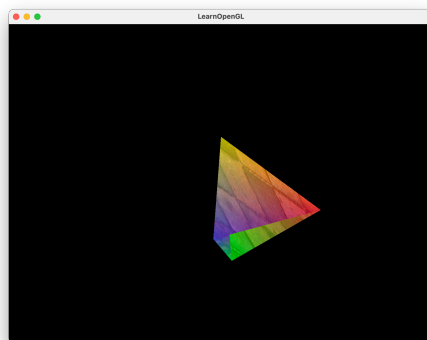


图 7: 实验 2 深度问题图 2

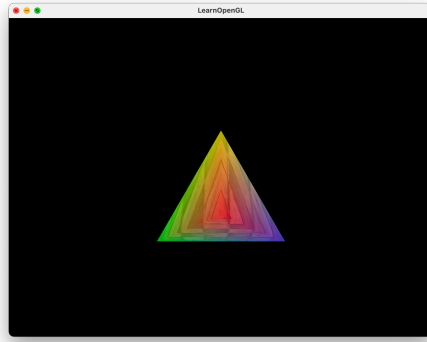


图 8: 实验 2 正面图

4 任务 3: 加入深度测试

4.1 任务要求

在任务 2 的基础上，增加深度测试，展示更具有真实感的立方体三视图。

4.2 关键代码

```
glEnable(GL_DEPTH_TEST);  
while (!glfwWindowShouldClose(window))  
{  
    processInput(window);  
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);  
    ...  
}
```

开启深度测试、清除深度缓冲即可。

4.3 结果图片

我们一共给出五个输出结果，分别是三视图和在旋转过程中的截图。

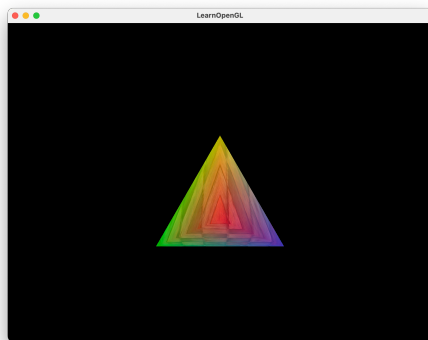


图 9: 实验 3 正视图

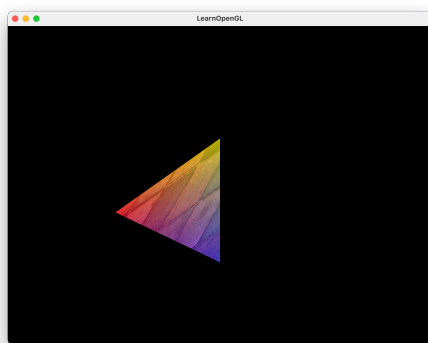


图 10: 实验 3 右视图

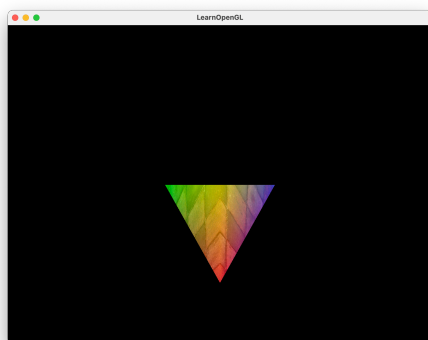


图 11: 实验 3 俯视图

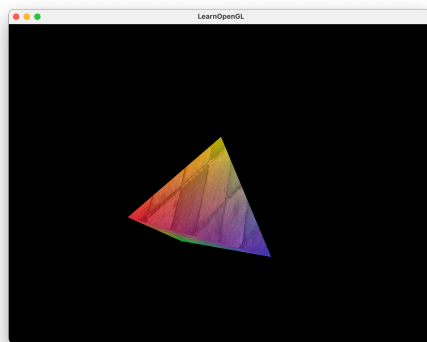


图 12: 实验 3 旋转过程中截图 1

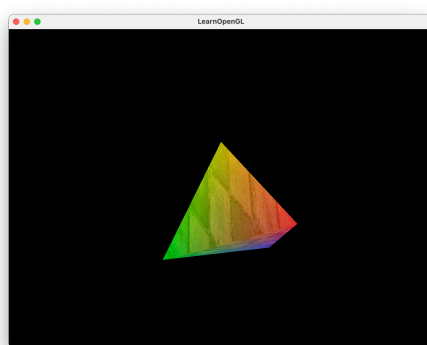


图 13: 实验 3 旋转过程中截图 2